CÓDIGOCLARO · RUTA REACT

Tema 06 · Listas y keys

Guía de estudio · DAM/DAW

map, keys estables y renderizado de colecciones.

1. Explicación

map devuelve una nueva colección y suele usarse para crear elementos JSX por cada dato. key ayuda a React a relacionar cada elemento con su identidad cuando una lista cambia. Debe ser estable y único entre hermanos, normalmente un id de los datos. El índice puede producir errores de estado visual si se insertan, eliminan o reordenan filas; no es automáticamente incorrecto para listas estáticas sin identidad propia, pero no es una solución universal.

2. Ejemplo

Código · jsx

```jsx
const temas = [{ id: 1, titulo: "Tipos" }, { id: 2, titulo: "Bucles" }];

function Lista() {
  return <ul>{temas.map(tema =>
    <li key={tema.id}>{tema.titulo}</li>
  )}</ul>;
}
```

Resultado esperado

```
Una lista con los elementos Tipos y Bucles.
```

3. Errores frecuentes

- Omitir key y recibir una advertencia que dificulta localizar cambios de identidad.
- Reutilizar la misma key para dos hermanos.
- Usar un índice como identidad cuando se pueden reordenar elementos con estado.

4. Ejercicio

Muestra los nombres de una lista de tareas y conserva una key asociada a cada tarea.

Casos que debe superar

Entrada	Resultado esperado
[]	Lista vacía
[{id:1,nombre:"A"}]	Un elemento A
Añadir una tarea antes de id 1	A mantiene su identidad y contenido

5. Solución explicada

Código · jsx

```jsx
function Tareas({ tareas }) {
  return <ul>{tareas.map(tarea =>
    <li key={tarea.id}>{tarea.nombre}</li>
  )}</ul>;
}
```

Criterio de aceptación: Se acepta otro identificador estable siempre que sea único entre hermanos y sobreviva a reordenamientos.

Otras soluciones válidas

- Crear elementos con map y una key id estable.
- Construir el array de elementos antes del return y renderizarlo después.

6. Fuentes oficiales

Esta guía resume conceptos para estudiar. Consulta las fuentes para comprobar detalles de versión y APIs.

Equipo de React — Rendering Lists